

온톨로지 기반 금융상품 질의응답 에이전트

신뢰 가능한 금융상품 탐색을 위한 의미 기반 Agent 시스템

팀명 입력

제10회 2026 미래에셋증권 AI Festival

2026년 9월

초록

본 제안서는 국내채권, 국내·해외 ETF, 공모펀드 등 이질적인 금융상품 데이터를 공통 의미 체계로 통합하고, 사용자의 자연어 질의를 온톨로지에 기반하여 해석한 뒤 관계 탐색, 수치 연산, 의미 검색을 적절한 저장소에 분배하여 검증 가능한 근거와 함께 답변하는 금융상품 Agent를 제안한다. 제안 시스템은 Canonical Data Model, Ontology, PostgreSQL, Knowledge Graph, Semantic Search, HyperCLOVA X를 결합하며, 검색 결과를 직접 생성 모델에 전달하지 않고 Evidence Validation을 거치는 Evidence-First 구조를 채택한다. 또한 데이터 범위나 비교 가능성이 충분히 검증되지 않은 경우 추정 답변을 생성하지 않는 Fail-Closed 전략을 통해 금융 도메인에서 요구되는 정확성, 재현성, 설명 가능성을 확보하는 것을 목표로 한다.

주요어: 금융상품 Agent, Ontology, Knowledge Graph, Federated Query, Entity Resolution, Evidence Validation, HyperCLOVA X

목차

1 제안 개요	4
1.1 제안 배경 및 문제 인식	4
1.2 제안 목표	5
1.3 핵심 아이디어	5
1.4 기존 방식과의 차별점	6
2 문제 정의	7
2.1 금융상품 데이터의 분산 및 이질성	7
2.2 상품 유형별 데이터 구조의 차이	7
2.3 의미적 불일치와 Entity/Relation 정합성 문제	7
2.4 LLM 단독 질의응답의 한계	7
2.5 Ontology 기반 의미 통합의 필요성	7
3 제안 시스템 개요	7
3.1 전체 Architecture	8
3.2 전체 데이터 및 질의 처리 흐름	9
3.2.1 사전 지식 구축	9
3.2.2 온라인 질의 처리	9
3.3 Data Layer	10
3.4 Knowledge Layer	11
3.4.1 Ontology	11
3.4.2 Knowledge Graph	11
3.4.3 Semantic Index	11
3.5 Agent Layer	12
3.6 API / Service Layer	12
4 금융상품 지식 모델링	15
4.1 Canonical Data Model	15
4.2 금융상품 Ontology 설계	15
4.3 주요 Entity	15
4.4 주요 Relation	15
4.5 금융 지표 및 속성 모델링	15
4.6 Entity Resolution 및 Relation Construction	15
4.7 Data-Ontology Mapping	15
4.8 외부 데이터 및 Provenance 관리	15
4.9 Ontology Constraint 및 정합성 검증	15
5 Agent 기반 질의 처리	15
5.1 Query Understanding	15
5.2 Entity Resolution	15
5.3 Ontology Grounding	15
5.4 Query Planning	15
5.5 RDB / Graph / Semantic Retrieval	15
5.6 Evidence Construction	15

5.7	Validation	15
5.8	HyperCLOVA X 기반 최종 응답 생성	15
6	주요 질의 유형 및 사용자 시나리오	15
6.1	조건 기반 금융상품 검색	15
6.2	상품 비교 및 Ranking	15
6.3	특정 기업 관련 금융상품 탐색	15
6.4	ETF 구성종목 기반 탐색	15
6.5	관계 기반 복합 질의	15
6.6	자연어 설명형 질의	15
6.7	데이터 부족·비교 불가 질의 처리	15
7	신뢰성 및 검증	15
7.1	Source Provenance	15
7.2	Snapshot 및 Version 관리	15
7.3	Ontology-Data 정합성	15
7.4	PostgreSQL-Neo4j 정합성	15
7.5	수치 및 Ranking 검증	15
7.6	Evidence Validation	15
7.7	Fail-Closed 전략	15
8	기대 효과 및 확장성	15
8.1	금융상품 탐색 편의성	15
8.2	근거 기반 답변 신뢰도 향상	15
8.3	금융상품 간 의미적 연결	15
8.4	신규 금융상품 및 데이터 공급자 확장	15
8.5	실시간 데이터 연계	15
8.6	사용자 맞춤형 Agent로의 발전	15
9	결론	15
A	용어 및 약어	16

1. 제안 개요

1.1 제안 배경 및 문제 인식

최근 금융서비스는 단순한 상품 정보 제공을 넘어, 분산된 금융 데이터를 통합하고 사용자의 상황과 요구에 맞는 정보를 제공하는 개인화 서비스 중심으로 확장되고 있다. 금융 마이데이터는 은행, 카드사 등 여러 기관에 분산된 개인 신용정보를 한곳에서 파악하고 관리함으로써 맞춤형 금융서비스를 합리적으로 이용할 수 있도록 지원하는 제도로 도입되었다. 최근 금융 마이데이터 시장에서도 개인화 서비스 강화가 향후 소비자 경쟁력을 좌우하는 주요 요소로 언급되고 있다 [빅데이터뉴스].

이러한 변화는 금융상품 탐색 또한 사용자가 개별 상품과 지표를 직접 검색하는 방식에서, 자연어로 자신의 요구를 제시하면 시스템이 필요한 상품 정보를 탐색·비교·연산하고 그 근거를 설명하는 방향으로 발전할 필요가 있음을 보여준다. 미래에셋증권의 과제설명자료 역시 금융상품 Agent의 핵심을 정형 금융상품 데이터를 Agent가 스스로 탐색·연산하고, 그 결과를 근거에 기반하여 답변하는 Agent RAG·QA로 제시한다. 이를 위해 상품 도메인 특화 Ontology, RDB·Vector·Graph 기반 KnowledgeBase, Intent Analysis & Retrieval Engine, Evidence 기반 Answer Generator를 주요 구성요소로 제시하고 있다 [과제설명자료, 4쪽].

그러나 금융상품에 대한 자연어 질의를 정확하게 처리하는 것은 일반적인 정보 검색보다 복잡하다. 국내채권, 국내 ETF, 해외 ETF, 공모펀드는 서로 다른 속성과 데이터 구조를 가지며, 하나의 사용자 질의 안에서도 상품 유형, 투자지역, 위험등급, 보수, 수익률과 같은 정형 조건뿐 아니라 기업 간 관계, ETF 편입 관계, 상품 간 비교 및 순위와 같은 서로 다른 형태의 정보 처리가 동시에 요구될 수 있다.

예를 들어 “특정 기업의 자회사를 편입한 ETF 중 순자산이 큰 상품을 알려줘”라는 질의는 단순한 키워드 검색으로 해결하기 어렵다. 먼저 기업과 자회사의 관계를 식별하고, 해당 자회사가 어떤 ETF에 편입되어 있는지를 관계 기반으로 탐색한 뒤, 각 ETF의 순자산을 조회하고 정렬해야 한다. 즉 하나의 자연어 질의에도 **의미 이해, Entity Resolution, 관계 탐색, 수치 연산, 근거 검색**이 복합적으로 요구된다.

이러한 특성 때문에 단일 검색 방식만으로는 복합 금융 질의를 안정적으로 처리하기 어렵다. 기술세션자료는 상품명, 보수율, 위험 정보가 서로 다른 문서 조각에 존재할 경우 일부 관련 문서만 검색되고 필요한 정보가 누락될 수 있는 *Fragmented Retrieval* 문제를 제시한다. 또한 복합 질의를 처리하기 위해서는 의미 유사도 검색, 관계 순회, 숫자 필터링을 질문의 성격에 따라 함께 활용해야 함을 설명한다 [기술세션자료, 19-21쪽].

문제는 단순히 검색 범위를 넓히는 것으로 해결되지 않는다. 기술세션자료의 국내 ETF 자체 벤치마크에서는 SQL 실행 성공률이 약 100%임에도 실제 정답률은 45%에 머물렀으며, 주요 실패 원인은 잘못된 컬럼, 값, 조건의 선택이었다. 반면 Ontology를 단순 프롬프트 설명이 아니라 런타임 식별 과정에 활용했을 때 더 큰 정확도 개선이 나타났고, RDB와 Graph를 결합한 방식이 단일 검색 방식보다 안정적인 결과를 보였다 [기술세션자료, 23쪽]. 이는 금융상품 질의응답에서 중요한 문제가 단순히 실행 가능한 SQL이나 유사한 문서를 찾는 것이 아니라, **사용자의 표현을 올바른 금융 개념, 관계, 데이터 필드에 연결하는 것**임을 보여준다.

또한 금융 도메인에서는 정확한 답변을 생성하는 것뿐 아니라 **근거가 부족한 경우 답변을 생성하지 않는 것** 역시 중요하다. 과제설명자료에서는 제공 데이터로 확인할 수 없는 답변 불가 질의가 평가에 포함되며, 해당 질의에 임의의 답변을 생성하면 감점한다고 명시한다. 반대로 “확인할 수 없음”을 명시하거나 답변에 필요한 조건을 역질문하는 것은 올바른 처리로 인정된다 [과제설명자료, 7쪽]. 또한 모든 답변은 데이터에 근거해야 하며, 근거 없는 수익률 전망이나 단정적 투자 추천은 금지된다 [과제설명자료, 6쪽].

따라서 본 제안이 해결하고자 하는 핵심 문제는 단순히 더 많은 금융상품을 검색하는 것이 아니다. 본 제안은 **자연어 질의를 금융 도메인의 명시적인 의미 체계에 Grounding하고, 질문에 필요한**

관계 탐색과 수치 연산을 적절한 데이터 소스에 배분하며, 그 결과를 검증 가능한 Evidence로 구성하는 것을 금융상품 Agent의 핵심 문제로 정의한다.

이를 위해 본 시스템은 Ontology를 중심 Semantic Layer로 활용하고, RDB, Knowledge Graph, Semantic Search를 결합한 Federated Retrieval과 Evidence Validation을 통해 검증 가능한 범위에서만 답변하는 신뢰 가능한 Financial Product Analyst를 제안한다.

1.2 제안 목표

본 제안의 목표는 이질적인 금융상품 데이터를 공통된 의미 체계로 통합하고, 사용자의 자연어 질의를 Ontology에 기반하여 해석한 뒤, 질의 특성에 가장 적합한 데이터 저장소를 선택·조합하여 검증 가능한 근거와 함께 답변하는 금융상품 Agent를 구축하는 것이다.

이를 위해 본 시스템은 단순한 LLM 기반 질의응답이나 단일 검색 시스템을 지향하지 않는다. 금융상품 데이터의 구조와 의미를 보존하는 Canonical Data Model을 구축하고, 그 위에 금융상품의 개념·속성·관계를 정의하는 Ontology를 배치한다. 이후 사용자의 질의를 의미 단위로 분석하여 관계 정보는 Graph, 정량적 조건과 연산은 RDB, 자연어 의미 및 설명 정보는 Semantic Search가 담당하도록 역할을 분리한다. 이는 기술세션자료에서 제시한 “Graph는 관계를 찾고, RDB는 숫자를 필터링하며, Vector는 의미를 검색한다”는 Federated Query의 역할 분담과 일치한다 [기술세션자료, 22쪽].

본 제안은 다음 네 가지 목표를 갖는다.

1. 금융상품 데이터의 의미적 통합

상품군별 서로 다른 원천 스키마를 그대로 Agent에 노출하는 대신, Canonical Data Model과 Ontology를 이용하여 동일한 개념과 관계를 일관된 의미로 표현한다.

2. 질의에 따른 동적 검색 및 연산

자연어 질의를 구조화된 실행 계획으로 변환하고, RDB·Graph·Semantic Search 중 필요한 저장소를 선택하여 복합 조건, 관계 탐색, 비교 및 설명을 수행한다.

3. 근거 중심의 신뢰 가능한 응답 생성

검색 결과를 즉시 LLM에 전달하지 않고, 출처, 데이터 기준, 엔티티 정합성, 수치 및 관계 조건을 검증하여 Evidence를 구성한 뒤 HyperCLOVA X가 이를 바탕으로 최종 응답을 생성하도록 한다.

4. 불확실성에 대한 보수적 처리

질의에 필요한 개념이나 관계가 지원되지 않거나, 데이터 범위가 부족하거나, 비교 가능성이 검증되지 않은 경우 추정에 의해 답을 생성하지 않고 해당 요청의 한계를 명시한다.

1.3 핵심 아이디어

본 시스템의 핵심 아이디어는 Ontology를 단순한 설명 문서가 아니라 질의 해석과 검색·검증을 통제하는 런타임 Semantic Layer로 활용하는 것이다. 기술세션자료는 Ontology를 “무엇이 가능해야 하는가”를 정의하는 TBox에 가까운 개념·규칙 계층, Knowledge Graph를 “실제로 무엇이 있는가”를 저장하는 ABox에 가까운 인스턴스 계층으로 구분한다 [기술세션자료, 4쪽]. 또한 Ontology가 용어 표준화, 관계 기반 추론, 데이터 무결성 검증, LLM 답변의 지식 그래프 Grounding을 지원할 수 있음을 설명한다 [기술세션자료, 5쪽].

본 제안은 이러한 Semantic Layer를 실제 Agent 실행 과정에 연결하기 위해 다음 네 단계의 구조를 사용한다.

이 흐름은 기술세션자료가 전체 구조를 Define → Connect → Retrieve → Reason으로 정리한 방향과 대응한다. 즉 Ontology로 개념과 관계를 정의하고, Knowledge Graph로 실제 데이터를

표 1: 제안 시스템의 핵심 처리 단계

단계	핵심 구성	설명
Define	Canonical Data Model & Ontology	서로 다른 금융상품 원천 데이터를 공통 모델로 정규화하고, 금융상품 도메인의 개념·속성·관계와 의미 제약을 정의한다.
Connect	Knowledge Construction	정규화된 금융상품과 외부 데이터를 Entity Resolution을 통해 동일한 실제 개체에 연결하고, 검증된 관계를 Knowledge Graph에 구성한다. 각 데이터에는 출처와 기준 시점 등의 Provenance를 함께 관리한다.
Retrieve	Ontology-Grounded Federated Query	사용자 질의를 Query Understanding, Entity Resolution, Ontology Grounding, Query Planning 과정을 통해 구조화한다. 수치·조건·정렬에는 RDB, 다단계 관계 탐색에는 Graph, 자연어 의미 탐색에는 Semantic Search를 활용한다.
Reason	Evidence-First Answer Generation	각 저장소의 결과를 바로 답변으로 변환하지 않고, Evidence Builder와 Validator를 통해 출처, 엔티티, 관계, 수치 및 비교 조건을 검증한다. 최종적으로 검증된 Evidence만 HyperCLOVA X에 제공한다.

연결하며, Federated Query로 질문에 필요한 근거를 찾고, LLM/GraphRAG가 근거를 바탕으로 답변한다 [기술세션자료, 24쪽].

따라서 본 시스템에서 LLM은 금융 지식의 유일한 저장소나 판단 주체가 아니라, **구조화되고 검증된 금융 지식을 사용자가 이해할 수 있는 언어로 연결하는 추론 및 표현 계층**으로 동작한다.

1.4 기존 방식과의 차별점

본 제안의 차별점은 단순히 RDB, Graph, Vector DB와 LLM을 함께 사용한다는 데 있지 않다. 핵심은 **하나의 Ontology와 Canonical Data Model을 중심으로 각 저장소의 역할을 명확히 분리하고, 자연어 질의를 이 의미 체계에 Grounding한 후 검증 가능한 실행 계획을 생성한다는 것**이다.

기술세션자료는 같은 LLM을 사용하더라도 문서 Chunk 중심 접근과 Ontology·Graph·RDB를 함께 사용하는 Semantic AI 접근은 지식 활용 방식이 다르다고 설명한다. 전자는 Embedding으로 관련 문서 조각을 검색하는 데 초점을 두는 반면, 후자는 Ontology 해석, Graph 탐색, RDB 검증·필터를 거쳐 의미·관계·검증 중심으로 답변을 구성한다 [기술세션자료, 16쪽].

본 시스템은 이러한 방향을 실제 실행 구조로 구체화하여 **Ontology를 Query Planning과 Validation의 공통 기준으로 사용한다**. 즉 Ontology는 단순히 LLM 프롬프트에 금융 용어 설명을 제공하는 보조 자료가 아니라,

- 자연어 질의가 무엇을 의미하는지 결정하고,
- 어떤 관계와 필드를 사용할 수 있는지 제한하며,
- 어떤 저장소에서 근거를 찾아야 하는지를 결정하고,
- 검색된 결과가 의미적으로 유효한지 검증하는

Agent의 런타임 Semantic Layer로 활용된다. 기술세션자료 역시 Ontology를 프롬프트 설명이 아니라 런타임 식별에 사용할 때 더 큰 성능 개선이 관찰되었으며, 이를 “문서가 아니라 런타임 통제 계층”으로 정리한다 [기술세션자료, 23쪽].

또한 본 제안은 “가능한 한 많은 답을 생성하는 시스템”보다 **검증할 수 있는 범위에서 정확하게**

표 2: 기존 접근과 제안 시스템의 비교

구분	문서 중심 RAG	단일 RDB / NL2SQL	제안 시스템
지식 표현	문서 · Chunk 중심	테이블 · 컬럼 중심	Canonical Model + Ontology
자연어 의미 해석	Embedding 유사도 중심	Schema / Column Mapping 중심	Ontology Grounding
정량 필터 · 정렬 다단계 관계 탐색	제한적 어려움	강점 JOIN 구조에 의존	RDB 기반 정확한 연산 Knowledge Graph 관계 탐색
설명 · 의미 검색 복합 질의	강점 검색 Chunk 결합	제한적 SQL 표현 범위에 의존	Semantic Search 활용 Query Decomposition + Federated Query
엔티티 통합	문자열 유사도 의존	Key 설계에 의존	Explicit Entity Resolution
답변 근거 의미적 제약 데이터 부족 처리	검색 Chunk 제한적 LLM 추론으로 보완될 위험	Query 결과 DB Schema 중심 빈 결과 중심	검증된 Evidence Bundle Ontology 기반 의미 제약 Validation + Fail-Closed
LLM 역할	검색 및 생성의 중심	NL2SQL 생성	검증된 근거에 대한 추론 · 설명

답하는 시스템을 지향한다. 데이터에 존재하지 않는 정보, 의미적으로 지원되지 않는 관계, 신뢰할 수 없는 비교 조건에 대해서는 LLM이 임의로 보완하지 않도록 하며, 이러한 Fail-Closed 전략을 통해 금융상품 탐색에서 요구되는 신뢰성과 설명 가능성을 확보한다. 이는 평가 데이터로 확인할 수 없는 질문에 답변을 생성하면 감점한다는 과제의 평가 원칙과도 직접 연결된다 [과제설명자료, 7쪽].

결과적으로 본 제안은 단순한 Question → Retrieval → LLM 구조를 넘어,

Question → Semantic Interpretation → Structured Planning → Federated Retrieval
→ Evidence Validation → Grounded Answer

의 전체 파이프라인을 구축한다. 이는 금융 데이터의 의미와 관계를 이해하고, 필요한 연산을 수행하며, 그 결과를 검증 가능한 근거와 함께 설명하는 **Financial Product Analyst**를 구현하기 위한 접근이다.

2. 문제 정의

2.1 금융상품 데이터의 분산 및 이질성

2.2 상품 유형별 데이터 구조의 차이

2.3 의미적 불일치와 Entity/Relation 정합성 문제

2.4 LLM 단독 질의응답의 한계

2.5 Ontology 기반 의미 통합의 필요성

3. 제안 시스템 개요

본 시스템은 금융상품 데이터를 단순히 하나의 데이터베이스에 적재하고 LLM이 직접 조회하도록 하는 구조가 아니라, 데이터의 의미를 Ontology로 명시하고, 질의의 성격에 따라 RDB, Knowledge Graph, Semantic Search를 선택적으로 활용한 뒤, 검증된 Evidence만을 기반으로 최종 답변을 생성하는 Ontology-Grounded Federated Agent Architecture로 설계하였다.

미래에셋증권 기술세션자료는 복합 금융 질의를 의미 단위로 분해한 후 Graph, RDB, Vector에 각각 전달하고, 검색된 결과를 통합·검증하여 답변하는 Federated Query Architecture를 제시한다. 특히 Graph는 관계 탐색, RDB는 숫자 필터링, Vector는 의미 검색에 각각 강점을 가진다고 설명한다 [기술세션자료, 21-22쪽]. 본 시스템은 이러한 방향을 실제 실행 가능한 소프트웨어 구조로 구체화하되, Ontology Grounding과 Evidence Validation을 각 실행 단계의 공통 제약으로 적용하였다.

3.1 전체 Architecture

제안 시스템은 크게 **Data Construction Layer**, **Semantic Knowledge Layer**, **Agent Execution Layer**, **Evidence & Answer Layer**, **Service Layer**의 다섯 계층으로 구성된다.

표 3: 제안 시스템의 전체 아키텍처

계층	주요 구성요소	역할
Data Construction	Source Data, Canonicalization, External Data	주최 측 금융상품 데이터와 외부 보완 데이터를 수집·정제하고, 서로 다른 상품군의 정보를 Canonical Data Model로 통합한다.
Semantic Knowledge	Ontology, PostgreSQL, Neo4j, Semantic Index	금융상품의 개념·관계·제약을 Ontology로 정의하고, 정량 데이터는 PostgreSQL, 관계 데이터는 Neo4j, 설명형 정보는 Semantic Index에 저장한다.
Agent Execution	Query Understanding, Entity Resolution, Ontology Grounding, Query Planning, Retriever / Executor	자연어 질문을 구조화된 의미 표현으로 변환하고, 필요한 저장소와 연산을 결정하여 QueryPlan을 실행한다.
Evidence & Answer	Evidence Builder, Validator, Safe Response, HyperCLOVA X	검색 결과를 출처·엔티티·수치·관계 단위로 검증하고, 검증된 Evidence만을 기반으로 최종 자연어 답변을 생성한다.
Service	FastAPI, Docker, Naver Cloud	평가 시스템과 연결되는 API를 제공하고, 애플리케이션 및 데이터 저장소의 실행·상태·배포를 관리한다.

전체 처리 구조를 요약하면 다음과 같다.

Offline Knowledge Construction

Source Data → Cleaning / Canonicalization → Ontology Mapping → PostgreSQL / Neo4j / Semantic Index

Online Agent Execution

Question → Query Understanding → Ontology Grounding → Query Planning → Federated Retrieval → Evidence Validation → HyperCLOVA X → Answer

이 구조의 핵심은 LLM이 원천 데이터에 직접 접근하여 모든 판단을 수행하도록 하지 않는다는 점이다. 질문의 의미 해석과 데이터 접근 사이에 Ontology 및 구조화된 QueryPlan을 두고, 검색 결과와 답변 생성 사이에는 Evidence Validation을 둔다. 따라서 자연어 생성 모델은 전체 시스템의 지식을 임의로 결정하는 주체가 아니라, 검증된 금융 지식을 사용자에게 전달하는 최종 추론·표현 계층으로 사용된다.

3.2 전체 데이터 및 질의 처리 흐름

본 시스템의 처리 과정은 사전 지식 구축 과정과 온라인 질의 처리 과정으로 구분된다.

3.2.1 사전 지식 구축

평가 질의 처리 중 원천 데이터를 즉시 수집하거나 변환하지 않고, 평가 이전에 금융상품 데이터를 정규화하고 검증된 KnowledgeBase를 구축한다.

1. Source Data Ingestion

국내채권, 국내 ETF, 해외 ETF, 공모펀드 데이터를 각 원천 스키마에 따라 파싱한다. 필요 시 금융상품과 관련된 외부 데이터를 별도의 Snapshot으로 수집한다.

2. Cleaning and Canonicalization

서로 다른 상품군에서 사용되는 필드명, 값 표현, 식별자를 정규화하고, 공통 Canonical Data Model의 Entity, Metric, Relation으로 변환한다.

3. Ontology Mapping

정규화된 데이터의 개념과 필드를 Ontology의 Class, Property, Relation에 연결한다. 이 과정에서 허용되지 않는 관계나 해석이 불명확한 데이터는 임의로 보완하지 않고 unresolved 상태 또는 source evidence로 보존한다.

4. KnowledgeBase Construction

정량적이고 정형적인 정보는 PostgreSQL에 저장하고, 상품·기업·증권·기관 간 검증된 관계는 Neo4j Knowledge Graph로 Projection한다. 자연어 기반 설명이나 의미 검색이 필요한 정보는 별도의 Semantic Index로 구축한다.

5. Provenance and Validation

Canonical Fact와 원천 데이터 사이의 Evidence Link를 보존하고, 데이터 Snapshot, Ontology Version 및 생성 버전을 함께 관리한다. 이를 통해 동일한 답변이 어떤 원천 정보에서 생성되었는지 추적할 수 있도록 한다.

현재 데이터 모델에서는 하나의 Canonical Fact에 복수의 원천 Evidence를 연결하되 동일한 사실을 중복 저장하지 않는 구조를 사용한다. 즉 Canonical Fact와 Source Record를 직접 혼합하지 않고 중간 Evidence Link를 통해 provenance를 유지한다.

3.2.2 온라인 질의 처리

사용자의 자연어 질의가 입력되면 다음 과정을 순차적으로 수행한다.

1. Query Reception

FastAPI의 /answer endpoint가 question_id와 question을 수신한다.

2. Query Understanding

질의에서 상품 유형, 필터 조건, 비교 조건, 정렬 기준, 관계 탐색 조건, 요청 필드를 추출한다. 명확하게 구조화 가능한 질의는 deterministic rule parser를 우선 활용하며, 규칙만으로 충분히 해석되지 않는 표현은 구성에 따라 HyperCLOVA X 기반 Semantic Parser의 보조를 받을 수 있도록 설계하였다.

3. Entity Resolution and Ontology Grounding

자연어 표현을 Canonical Entity와 Ontology Concept으로 변환하고, 사용 가능한 Field와 Relation을 결정한다. 예를 들어 자연어의 “순자산”은 canonical AUM field에, 특정 운용 관계는 Ontology의 관계 property에 연결된다.

4. Query Planning

Grounding된 Semantic Query를 구조화된 QueryPlan으로 변환한다. 현재 Planner는 deterministic하게 동작하며, 각 조건을 RDB, Graph, Semantic Retrieval 중 어떤 실행 경로에서 처리할지 결정한다.

5. Federated Retrieval

RDB Retriever는 수치 조건, 필터, 정렬 및 Ranking을 담당하고, Graph Retriever는 다단계 관계를 탐색하며, Semantic Retriever는 자연어 의미 및 설명 기반 검색을 수행한다.

6. Evidence Construction

각 Retriever의 결과를 단순히 합치는 것이 아니라 사용된 데이터, Entity, Relation, Metric, Source 및 Snapshot을 Evidence 형태로 구성한다.

7. Validation

Evidence가 원래 질의의 모든 조건을 만족하는지 확인하고, 의미적으로 지원되지 않는 조건, 데이터가 존재하지 않는 조건, 비교 가능성이 보장되지 않는 조건을 검사한다.

8. Answer Generation

검증을 통과한 Evidence를 HyperCLOVA X에 제공하여 최종 자연어 답변을 생성한다. 충분한 Evidence가 확보되지 않은 경우에는 부분적으로 추정하여 답하지 않고 확인 불가 또는 지원하지 않는 질의로 처리한다.

9. API Response

최종적으로 평가 명세에 맞추어 `question_id`, `question`, `retrieved_context`, `think_trace`, `answer` 필드를 반환한다.

본 구조는 지원되지 않는 조건을 조용히 제거한 뒤 나머지 조건만 실행하는 방식을 허용하지 않는다. 지원하지 않는 의미가 존재할 경우 해당 QueryPlan을 명시적으로 차단함으로써, 질문의 일부만 만족하는 결과가 마치 전체 질의의 답인 것처럼 반환되는 *silent semantic loss*를 방지한다.

3.3 Data Layer

Data Layer의 목적은 서로 다른 구조를 갖는 금융상품 데이터를 Agent가 일관된 방식으로 탐색할 수 있는 Canonical Data로 변환하는 것이다.

주최 측에서 제공한 데이터는 국내채권, 국내 ETF, 해외 ETF, 공모펀드로 구성되며, 상품군마다 제공되는 속성과 데이터 grain이 서로 다르다. 따라서 원천 데이터를 하나의 flat table로 강제로 병합하지 않고, 원천의 정보와 상품별 grain을 보존하면서 공통적으로 해석할 수 있는 Canonical Data Model을 구성하였다.

현재 구현에서 정형 데이터의 주요 저장소는 PostgreSQL이다. Canonical v2 Schema는 원천 Dataset, Dataset Snapshot, Canonical Entity, Metric Observation, Relation 및 Evidence 정보를 분리하여 관리한다. 특히 데이터가 단순히 현재 값만 저장하는 것이 아니라, 어떤 Source와 Snapshot에서 유래했는지를 함께 보존하도록 설계하였다.

Data Layer에서 외부 데이터는 주최 측 데이터와 동일한 권한으로 무조건 병합되지 않는다. 각 외부 데이터는 별도의 Source 및 Snapshot으로 관리하며, 평가 기준 데이터와 충돌하는 경우에는 주최 측 제공 데이터가 우선하도록 구성한다. 이는 과제설명자료에서 제시한 데이터 활용 원칙과도 일치한다 [과제설명자료, 6쪽].

또한 production runtime에서는 검증된 Snapshot만을 사용하도록 Dataset 상태와 reconciliation 결과를 readiness 조건으로 확인한다. 이를 통해 개발 중간 상태의 데이터나 Ontology와 호환되지 않는 Snapshot이 온라인 질의 처리에 사용되는 것을 방지한다.

3.4 Knowledge Layer

Knowledge Layer는 원천 데이터에 존재하는 값을 단순히 저장하는 계층이 아니라, **금융 데이터가 무엇을 의미하며 어떤 관계가 허용되는지를 명시하는 Semantic Layer**이다.

본 시스템의 Knowledge Layer는 크게 Ontology, Knowledge Graph, Semantic Index의 세 요소로 구성된다.

3.4.1 Ontology

Ontology는 금융상품, 증권, 기업, 기관, 상품분류 및 각종 금융 개념을 Class로 정의하고, 이들 사이에서 허용되는 Relation과 Property를 명시한다. 현재 runtime에서는 RDF 기반 Ontology Service를 사용하여 자연어 표현을 Concept, Field, Relation으로 Grounding한다.

Ontology Service는 단순한 alias lookup 외에도 다음 역할을 수행한다.

- 자연어 표현의 Canonical Concept Resolution
- Ontology Property와 Canonical Field 간 Mapping
- Relation Alias Resolution
- Relation의 Domain / Range Compatibility 검증
- Query에서 사용 가능한 Relation 탐색

예를 들어 Ontology의 특정 AUM Property는 Planner가 사용하는 Canonical Field로 연결되고, 이 Canonical Field는 다시 실제 PostgreSQL Column으로 연결된다. 즉,

Ontology Meaning → Canonical Field → Physical Storage

의 세 계층을 분리하였다. 이러한 구조는 Ontology에 데이터베이스의 물리적 구현 세부사항을 직접 포함시키지 않으면서도, Agent가 자연어 의미에서 실제 검색까지 일관되게 연결할 수 있도록 한다.

3.4.2 Knowledge Graph

Neo4j는 금융 Entity 사이의 관계 탐색을 담당한다. RDB에서 여러 JOIN을 통해 표현해야 하는 다단계 관계를 Graph에서는 명시적인 Edge Path로 탐색할 수 있다.

예를 들어

기업 → 자회사 → ETF 편입 → 운용사

와 같은 질의는 관계의 방향과 대상 Entity를 이용해 Graph Query로 실행할 수 있다.

Graph에는 모든 원천 데이터를 임의로 Projection하지 않고, Ontology의 Domain / Range와 provenance 조건을 만족한 검증된 관계만을 적재하는 것을 원칙으로 한다. 따라서 Ontology는 Graph Schema의 의미적 계약으로도 활용된다.

3.4.3 Semantic Index

Semantic Index는 RDB의 정량 조건이나 Graph의 명시적인 관계만으로 표현하기 어려운 설명형 질의를 처리하기 위해 사용한다. 자연어 표현 및 관련 설명을 검색하되, 가능한 경우 product identifier 등의 Canonical Metadata와 결합하여 구조화된 검색 결과와 동일 Entity 범위에서 사용할 수 있도록 설계하였다.

따라서 세 Knowledge Source의 역할은 다음과 같이 구분된다.

RDB = 값과 연산 Graph = 관계 Semantic Index = 의미와 설명

이러한 역할 분담은 기술세션자료에서 제시한 Federated Query의 기본 원칙과 대응한다 [기술세션자료, 20-22쪽].

3.5 Agent Layer

Agent Layer는 자연어 질문을 받아 Knowledge Layer의 여러 저장소를 조합하고, 검증 가능한 답변까지 생성하는 실행 계층이다.

현재 구현은 하나의 거대한 LLM Agent가 모든 작업을 자율적으로 수행하는 방식이 아니라, 각 단계의 책임을 명시적으로 분리한 **structured agent pipeline**을 사용한다.

표 4: Agent Layer의 주요 구성요소

구성요소	주요 구현 영역	역할
Query Understanding	app/query	질의를 구조화하고 상품 유형, 조건, 정렬, 관계, 요청 필드 등을 추출한다.
Ontology Grounding	app/ontology	자연어 표현을 Canonical Concept, Field, Relation으로 변환하고 의미 호환성을 검사한다.
Query Planner	app/planning	Grounding된 의미를 QueryPlan으로 변환하고 RDB, Graph, Semantic 경로를 결정한다.
Retriever / Executor	app/retrieval, app/graph, app/execution	SQL filter · sort · ranking, Graph relation traversal, Semantic retrieval을 실행한다.
Evidence Validation	app/evidence	검색 결과가 질의 조건과 의미 제약을 충족하는지 검증하고 지원하지 않는 의미를 차단한다.
Agent Service	app/agent	각 구성요소를 하나의 실행 pipeline으로 orchestration하고 최종 Answer Generator에 Evidence를 전달한다.

이 구조에서 Query Planning은 deterministic하게 수행된다. 즉 LLM이 매 질의마다 임의의 Tool 사용 순서를 자유롭게 생성하도록 하지 않고, Grounding된 Query Semantic Model에 따라 허용된 실행계획만 생성한다.

반면 자연어 해석이 deterministic rule만으로 충분하지 않은 경우에는 HyperCLOVA X 기반 Semantic Parser를 보조적으로 사용할 수 있으며, 최종 답변 생성 단계에서도 HyperCLOVA X를 이용해 검증된 Evidence를 사용자 친화적인 자연어로 변환한다.

이와 같이 LLM과 deterministic component의 역할을 분리함으로써, 자연어 이해 및 표현 능력은 활용하면서도 수치 계산, 관계 해석, 데이터 선택과 같은 핵심 실행은 검증 가능한 구조화된 표현을 통해 통제하도록 설계하였다.

3.6 API / Service Layer

Service Layer는 위의 Agent Pipeline을 평가 환경에서 안정적으로 호출할 수 있도록 제공하는 계층이다.

현재 애플리케이션은 FastAPI를 기반으로 구성되며, 평가를 위한 핵심 Endpoint는 다음과 같다.

- GET /answer: 금융상품 질의 처리

- GET /live: API Process의 liveness 확인
- GET /health: 데이터 저장소 및 runtime readiness 확인

과제 명세에 따라 /answer는 question_id와 question을 입력으로 받고, 항상 다음 다섯 개의 문자열 필드를 반환한다 [과제설명자료, 11쪽].

```
{
  "question_id": "...",
  "question": "...",
  "retrieved_context": "...",
  "think_trace": "...",
  "answer": "..."
}
```

여기서 retrieved_context에는 답변 생성에 활용한 근거 데이터가 포함되고, think_trace에는 내부 추론 원문이 아니라 질의 해석, 검색, 필터링 및 검증과 같은 **운영상의 처리 단계와 도구 사용 요약**을 기록한다.

Production 환경에서는 Agent API, PostgreSQL, Neo4j를 Docker Compose 기반의 독립 Service로 구성한다. 각 Service는 동일한 backend network를 통해 통신하며, API Container는 PostgreSQL과 Neo4j에 service name 기반으로 접근한다.

또한 데이터 Artifact와 애플리케이션 코드를 분리하여, Ontology, 외부 데이터 Snapshot, Semantic Index 등의 Knowledge Artifact는 checksum과 version을 통해 고정하고, Agent Application은 Git commit 단위의 Docker Image로 배포하도록 구성하였다. 이를 통해 코드 변경과 데이터 변경을 독립적으로 추적할 수 있으며, 서버 시작 시 version compatibility와 health condition을 확인한 후에만 평가 API를 활성화하도록 한다.

결과적으로 Service Layer는 단순한 HTTP wrapper가 아니라, **동일한 데이터와 Ontology를 사용한 재현 가능한 실행 환경을 보장하고, 불완전한 runtime 상태에서 답변이 생성되는 것을 방지하는 production safety boundary**의 역할을 수행한다.

- 4. 금융상품 지식 모델링
 - 4.1 Canonical Data Model
 - 4.2 금융상품 Ontology 설계
 - 4.3 주요 Entity
 - 4.4 주요 Relation
 - 4.5 금융 지표 및 속성 모델링
 - 4.6 Entity Resolution 및 Relation Construction
 - 4.7 Data–Ontology Mapping
 - 4.8 외부 데이터 및 Provenance 관리
 - 4.9 Ontology Constraint 및 정합성 검증
- 5. Agent 기반 질의 처리
 - 5.1 Query Understanding
 - 5.2 Entity Resolution
 - 5.3 Ontology Grounding
 - 5.4 Query Planning
 - 5.5 RDB / Graph / Semantic Retrieval
 - 5.6 Evidence Construction
 - 5.7 Validation
 - 5.8 HyperCLOVA X 기반 최종 응답 생성
- 6. 주요 질의 유형 및 사용자 시나리오
 - 6.1 조건 기반 금융상품 검색
 - 6.2 상품 비교 및 Ranking
 - 6.3 특정 기업 관련 금융상품 탐색
 - 6.4 ETF 구성종목 기반 탐색
 - 6.5 관계 기반 복합 질의
 - 6.6 자연어 설명형 질의
 - 6.7 데이터 부족·비교 불가 질의 처리
- 7. 신뢰성 및 검증
 - 7.1 Source Provenance
 - 7.2 Snapshot 및 Version 관리
 - 7.3 Ontology–Data 정합성
 - 7.4 PostgreSQL–Neo4j 정합성
 - 7.5 수치 및 Ranking 검증
 - 7.6 Evidence Validation
 - 7.7 Fail-Closed 전략
- 8. 기대 효과 및 확장성

[과제설명자료] 미래에셋증권, 「제10회 2026 미래에셋증권 AI Festival 과제 소개 - 금융상품 Agent (참가자공유용)」, 2026.07.

[기술세션자료] 이우람, 「2026 미래에셋증권 AI Festival Technical Session」, 미래에셋증권 AI Engineering 본부, 2026.

[빅데이터뉴스] 임예린, 「미래에셋증권, 금융 마이데이터 브랜드평판 9월 1위...현대카드·하나은행 순」, 빅데이터뉴스, 2026.09.01.

A. 용어 및 약어

Ontology 금융상품 도메인의 개념, 속성, 관계 및 의미 제약을 정의하는 지식 모델

Knowledge Graph

Ontology 구조 위에 실제 엔티티와 관계 데이터를 연결한 그래프

Canonical Data Model

서로 다른 원천 데이터 구조를 공통 의미 체계로 정규화한 데이터 모델

Federated Query

질의를 여러 저장소에 분해·라우팅하고 결과를 통합하는 검색 방식

Evidence Bundle

최종 답변 생성을 위해 검증된 근거를 구조화한 묶음

Fail-Closed

충분한 근거가 없을 때 추정 답변 대신 명시적으로 응답을 제한하는 전략